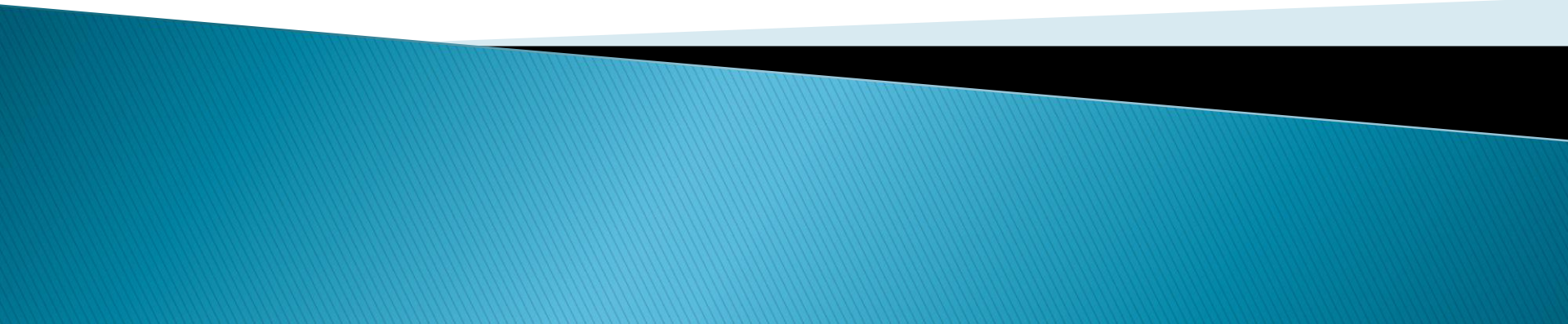


Алгоритмические стратегии



Алгоритмические стратегии

- ▶ **Стратегия** – детальный план, предназначенный обеспечить достижение цели
- ▶ **Алгоритм** – последовательность команд, предназначенная исполнителю, в результате выполнения которой он должен решить поставленную задачу

Алгоритмические стратегии

- ▶ Многие комбинаторные задачи можно представить как поиск элементов конечного множества, удовлетворяющих определенным условиям
- ▶ Перебор всех элементов множества с проверкой соответствующих условий даже для простых задач может потребовать огромных затрат

Полный перебор

Полный перебор

- ▶ Суть – проверяются все возможные варианты исходных данных на условие задачи. Если вариант удовлетворяет, то решение найдено
- ▶ Главное ограничение – неэффективность. Как правило, число возможных вариантов растет экспоненциально с увеличением размера задачи

Полный перебор

- ▶ Пример «Магический квадрат». Заполнить таблицу 3×3 девятью различными числами от 1 до 9 так, чтобы сумма чисел в каждой строке, каждом столбце и двух главных диагоналях была одинакова
- ▶ Количество вариантов:
 $9! = 9 \cdot 8 \cdot 7 \cdot 6 \cdot 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1 = 362\,880$
- ▶ Для таблицы 5×5 – $(5^2)! \cong 1,5 \cdot 10^{25}$
- ▶ Вопрос. Сколько времени потребуется компьютеру, выполняющему 100 триллионов операций в секунду, для решения этой задачи?

a	b	c
d	e	f
g	h	i

- ▶ $(1+2+\dots+9)/3=15$
- ▶ $(d+e+f)+(b+e+h)+(a+e+i)+(g+e+c)=$
 $=3\bullet e+(a+b+c)+(d+e+f)+(g+h+i)=$
 $=3\bullet e+3\bullet 15=4\bullet 15$
- ▶ $e=5$
- ▶ Пары с суммой 10: (1, 9), (2, 8), (3, 7), (4, 6)

1		
	5	
		9

- ▶ Такой вариант не проходит – нужно дважды набрать сумму 14 (по горизонтали и вертикали), а это возможно из
 - $9 + 5$ (цифры уже заняты)
 - $7 + 7$ (повтор цифры)
 - $8 + 6$ – только один возможный вариант, а надо 2

	1	
	5	
	9	

1	5	9

	9	
	5	
	1	

9	5	1

- ▶ В строке (столбце) с 1 числа 6 и 8
- ▶ Избежали полного перебора

Разделяй и властвуй

Стратегия «разделяй и властвуй»

- ▶ (метод декомпозиции) заключается в том, чтобы разделить задачу на несколько более легких подзадач (обычно такого же или похожего типа и желательно одного размера), решить каждую из них и, если это необходимо, скомбинировать их решения, чтобы получить решение исходной задачи
- ▶ Эта стратегия лежит в основе многих эффективных алгоритмов, использующихся для решения важных задач в информатике
- ▶ *Примеры:* сортировка слияниями, быстрая сортировка, бинарный поиск

Стратегия «разделяй и властвуй»

- ▶ **Варианты стратегии:**
 - «уменьшай и властвуй»
 - «преобразуй и властвуй»
- ▶ **Вариант «Уменьшай и властвуй»** основан на нахождении соотношения между решением данной задачи и решением её более простого частного случая
- ▶ **Пример:** сортировка вставками

Стратегия «разделяй и властвуй»

- ▶ Вариант «Преобразуй и властвуй» основан на идее преобразования. Задача решается в два этапа. Первоначально задача модифицируется или преобразуется в другую задачу, которую легче решить в силу тех или иных причин. На следующем этапе «властвования» она решается.
- ▶ Пример. Анаграммы (слова, состоящие из одних и тех же букв). Дан текст. Найти все анаграммы.
- ▶ Каждому слову сопоставим ключ – состоит из букв слов, отсортированных в алфавитном порядке. Сортируем слова по ключам. Анаграммы будут стоять рядом (воз – зов, атлас – салат)

Перебор с возвратом

Перебор с возвратом

- ▶ Перебор, но не полный, часть вариантов отсекается на основании некоторых критериев, вытекающих из условия задачи

Перебор с возвратом

Ключевой принцип – построение вариантов решения по принципу «один компонент за один раз» и оценка частично полученных вариантов решения по схеме:

- ▶ если частично построенный вариант не нарушает ограничений задачи, то подбирается первый из допустимых вариантов для следующего компонента решения
- ▶ если для следующего компонента нет допустимого варианта, то нет необходимости рассматривать для какого-то из оставшихся компонентов
- ▶ В этом случае алгоритм возвращается на предыдущий шаг и заменяет последний компонент частично построенного решения на следующий из возможных вариантов

Общая схема перебора с возвратом

- ▶ Даны n упорядоченных множеств $U_1, U_2, \dots, U_n$ и требуется построить вектор $A = (a_1, a_2, \dots, a_n)$, где $a_1 \in U_1, a_2 \in U_2, \dots, a_n \in U_n$, удовлетворяющий заданному множеству условий и ограничений

Общая схема

- ▶ Вектор A строится покомпонентно слева направо
- ▶ Предположим, что уже найдены значения первых $k-1$ компонент:
 $A=(a_1, a_2, \dots, a_{k-1}, ?, \dots, ?)$, тогда заданное множество условий ограничивает выбор следующего компонента a_k некоторым множеством $S_k \subset U_k$

Общая схема

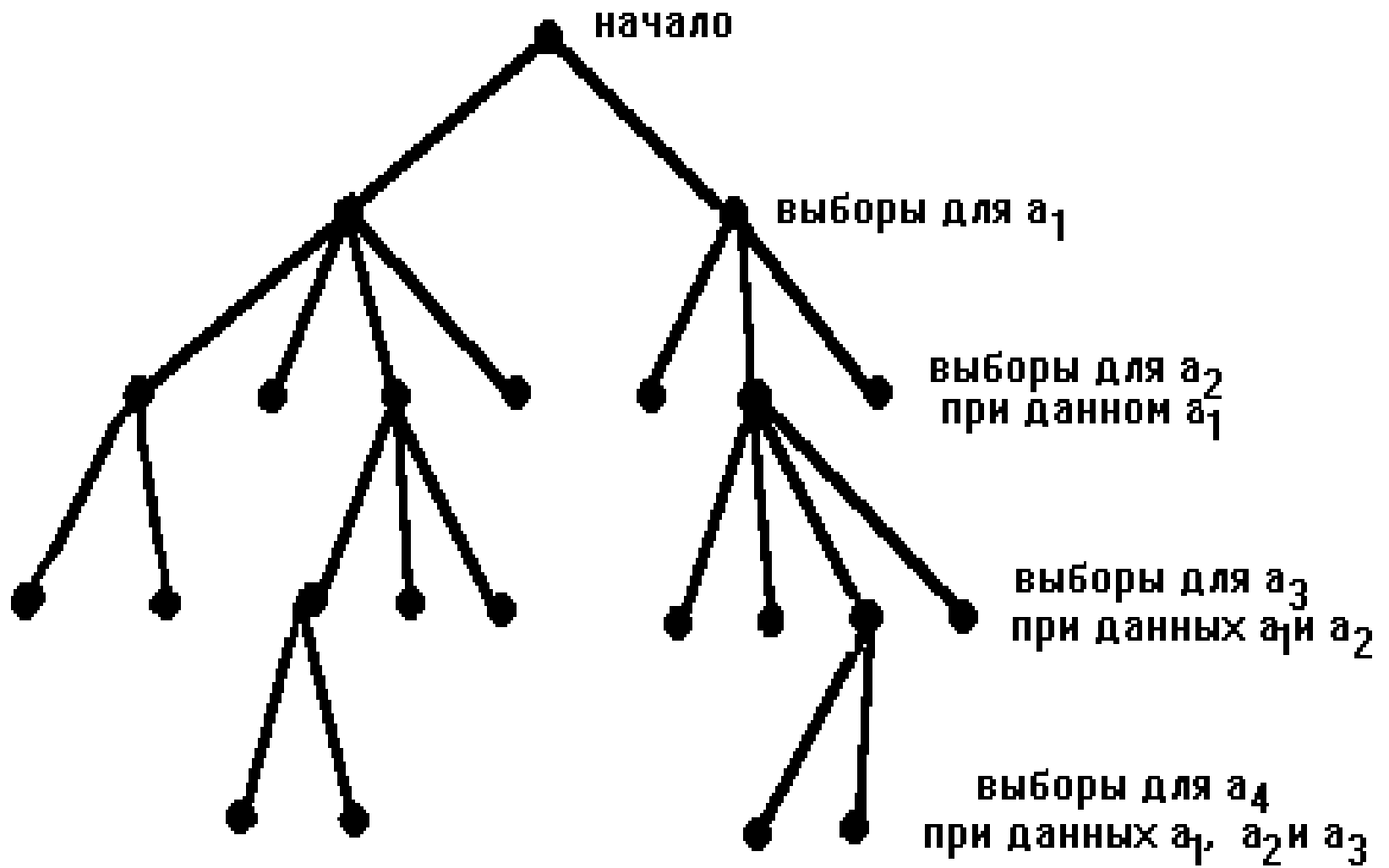
- ▶ Если S_k оказалось непустым, мы вправе выбрать в качестве a_k наименьший элемент S_k и перейти к выбору $(k+1)$ -го компонента и т. д.
- ▶ Если S_k оказалось пустым, то мы возвращаемся к предыдущему этапу (выбору $(k-1)$ -го компонента), отбрасываем $(k-1)$ -й элемент a_{k-1} и выбираем в качестве нового a_{k-1} тот элемент S_{k-1} , который непосредственно следует за только что отброшенным

Общая схема

- ▶ Может оказаться, что для нового a_{k-1} условия задачи допускают непустое S_k , и тогда мы попытаемся снова выбрать элемент a_k
- ▶ Если невозможно выбрать a_{k-1} , то возвращаемся еще на шаг назад и выбираем новый элемент a_{k-2} и т. д.

Графическое представление

- ▶ Графически этот процесс можно представить в виде **дерева поиска**
- ▶ Корень дерева (0 уровень) является пустым вектором
- ▶ Его сыновья – это множество кандидатов для выбора a_1
- ▶ В общем случае, узлы k -того уровня являются кандидатами для выбора a_k при условии, что $a_1, a_2, \dots, a_{(k-1)}$ выбраны так, как указывают предки этих узлов
- ▶ Вопрос о том, имеет ли задача решение, равносильен вопросу, являются ли какие-нибудь узлы дерева решениями. Разыскивая все решения, мы хотим получить все такие узлы



Последовательность действий

- ▶ Шаг 1. [Начало] Положить $k=1$ и $S_1 = U_1$
- ▶ Шаг 2. [Следующий элемент S_k] Если S_k пусто, то перейти к шагу 5, иначе положить a_k равным наименьшему из элементов S_k

Последовательность действий

- ▶ Шаг 3. [Найдено ли решение?] Если $k < n$, то перейти к шагу 4, иначе записать $(a_1, a_2, \dots, a_n)$ как решение. Если необходимо найти все возможные решения, то положить $k = k + 1$ и перейти к шагу 5, иначе остановиться
- ▶ Шаг 4. [Увеличение k] Увеличить k на 1, вычислить S_k и вернуться к шагу 2

Последовательность действий

- ▶ Шаг 5. [Возврат] Если $k=1$, то дальнейшее продвижение назад невозможно, поэтому остановиться; при этом либо найдены все решения, либо их не существует. Если $k>1$, то уменьшить k на 1, положить $S_k = S_k \setminus \{a_k\}$ и вернуться к шагу 2

Рекурсивная схема реализации алгоритма

{Глобальные переменные: величины, описывающие решение $(a_1, a_2, \dots, a_n)$, и множества $A_1, A_2, \dots, A_n$ }

procedure BackTrack (i):

if <решение найдено>: <записать или вывести решение>
else

 <сформировать S_i – множество допустимых значений из A_i ,
 $S_i \subset A_i$ >

 for $\langle s_i \in S_i \rangle$ Do:

 <включить s_i в решение>

 BackTrack (i+1)

 <исключить s_i из решения>

Оценка временной сложности алгоритма

- ▶ Данная схема реализации перебора приводит к экспоненциальным алгоритмам. Действительно, пусть все решения имеют длину N , тогда исследовать требуется порядка $|S_1| * |S_2| * \dots * |S_n|$ узлов дерева. Если значение S_i ограничено некоторой константой C , то получаем порядка C^n узлов

Пример 1. Задача о расстановке ферзей

- ▶ *Формулировка задачи*
- ▶ Существуют способы расстановки 8 ферзей на шахматной доске 8×8 так, чтобы они не били друг друга. Составить программу, которая находит один из таких способов
- ▶ На шахматной доске $n \times n$ требуется расставить n ферзей, не атакующих друг друга. Пусть задача заключается в поиске всех возможных расстановок ферзей

Пример 1. Задача о расстановке ферзей (n=4)

- ▶ *Первый вариант.* Всего клеток 16 (длина вектора ответа A). В 4 из них стоит один ферзь. Сокращение перебора определяется только тем фактом, что на одну клетку поля нельзя ставить двух и более ферзей.
- ▶ Число способов расстановки (сочетания без повторений) $= \frac{16!}{4! \cdot 12!} = 1\,820$ вариантов
- ▶ Для n=8 количество вариантов
4 426 165 368

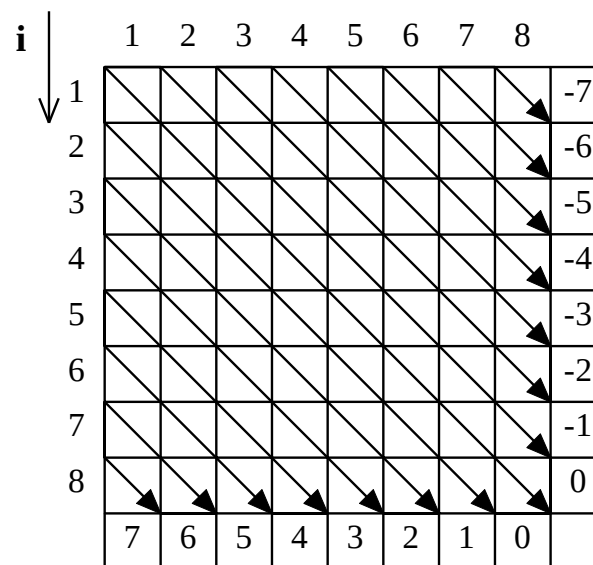
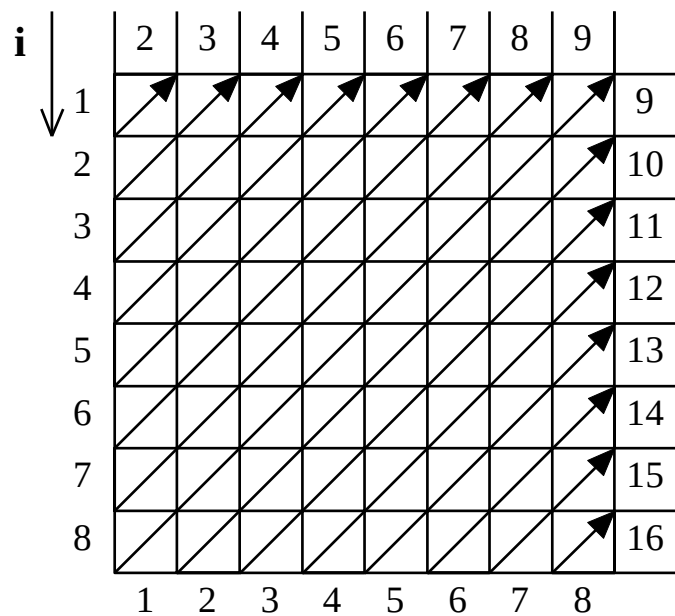
Пример 1. Задача о расстановке ферзей

- ▶ *Второй вариант.* Каждый столбец поля может содержать только одного ферзя. В каждом A_i уже не 16 элементов, а 4, что существенно сокращает перебор. Таких расстановок 4^4 или 256 (для $n=8$ – 16 777 216)
- ▶ *Третий вариант.* Никакие два ферзя нельзя поставить на одну горизонталь. Число расстановок $4!=24$ (для $n=8$ – 40 320)

Пример 1. Задача о расстановке ферзей

- ▶ *Четвертый вариант.* Количество элементов в A_i равно 4. Однако, если учитывать факт того, что на одну диагональ можно ставить только одного ферзя, то постановка ферзя на каждую горизонталь с номером i существенно сокращает количество допустимых вариантов на горизонтали с номером $i+1$

► Структуры данных

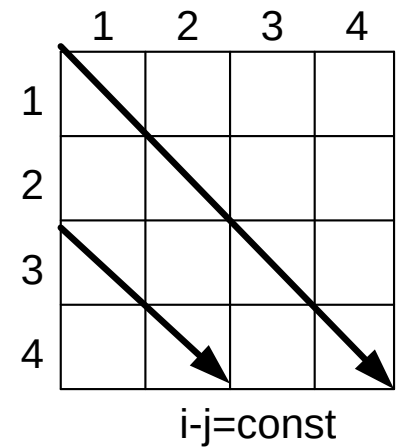
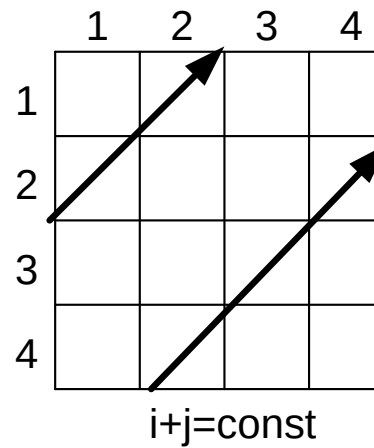
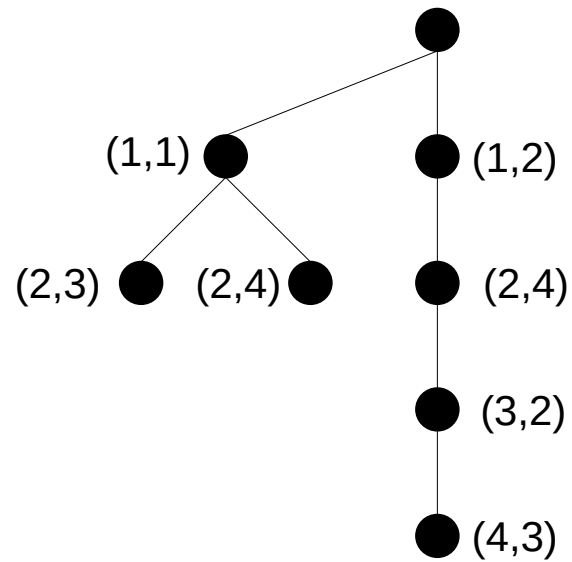


- ▶ Пример для поля 4×4

	1	2	3	4
1	1	@	@	@
2	@	@	2	#
3	@	#	@	#
4	@			@

	1	2	3	4
1	1	@	@	@
2	@	@	#	2
3	@	3	@	#
4	@	#	\$	@

	1	2	3	4
1	@	1	@	@
2	@	@	@	2
3	3	@	#	@
4	\$	@	4	#



Общая схема решения

Процедура solve(i):

{Вход: номер компоненты i}

{Выход: значение компоненты, переход к следующей, если оно найдено, или возврат к предыдущей компоненте}

если <решение найдено>: <вывод решения>

иначе:

<сформировать S_i из A_i , $S_i \subset A_i$ >

для $\langle s_i \in S_i \rangle$:

<включить s_i в решение>

solve(i+1)

<исключить s_i из решения >

Структуры данных

k: Integer;

x: Array [1..n] Of Integer; {решение}

a: Array [1..n] Of Boolean; {вертикаль}

b: Array [2..2 * n] Of Boolean; {восходящая
диагональ(/)}

c: Array [-n + 1..n - 1] Of Boolean; {нисходящая
диагональ(\)}

Инициализация

```
def init()  
    k = 0  
    for i = 1 To n: a[i] = True  
    for i = 2 To 2 * n: b[i] = True  
    for i = 1 - n To n - 1: c[i] = True
```

Действия

{процедура – сделать ход}

def move(i, j: Integer): #i,j – номер клетки

 x[i] = j

 a[j] = False; b[i + j] = False; c[i - j] = False;

{процедура отмены хода}

def back_move(i, j: Integer): #i,j – номер клетки

 a[j] = True; b[i + j] = True; c[i - j] = True;

Проверка хода

{функция проверки, можно ли сделать ход}

```
bool D_hod(i, j: Integer): #i,j – номер клетки  
    return a[j] and b[i + j] and c[i - j]
```

Основная процедура перебора

```
def solve(i: Integer):  
    j = 0  
    repeat  
        j++; q = False //Inc(k)  
        if d_hod(i, j) :  
            move(i, j)  
            if i < n:  
                solve(i + 1)  
            if not q : back_move(i, j)  
        else q = True  
    until q Or (j == n)  
    if q : print
```

Вывод ответа

F	*	*	*	*	*	*	*
*	*	*	*	F	*	*	*
*	*	*	*	*	*	*	F
*	*	*	*	*	F	*	*
*	*	F	*	*	*	*	*
*	*	*	*	*	*	F	*
*	F	*	*	*	*	*	*
*	*	*	F	*	*	*	*

[1, 5, 8, 6, 3, 7, 2, 4]